

AI Career Compass

Frontend — Complete Technical & Design Documentation
Career Guidance System Project · React + Vite · 2024

5 Pages

React + Vite

FastAPI Connected

Ocean UI Theme

CSS Animations

24 Careers Covered

TABLE OF CONTENTS

- 1. Project Overview
- 2. Tech Stack & Tools
- 3. Application Structure & File Layout
- 4. Page-by-Page Breakdown
- 5. Reusable Components
- 6. Design System — Colours, Fonts & Animations
- 7. API Integration with FastAPI Backend
- 8. Frontend Data Logic
- 9. State Management & Page Navigation
- 10. How to Run the Full Project
- 11. Likely Reviewer Questions & Answers
- 12. Known Limitations & Future Improvements

01

Project Overview

AI Career Compass is a fully responsive, animated single-page React web application that serves as the frontend interface for the Career Guidance System. It collects a student's academic background, skills, and interests, sends them to a FastAPI machine learning backend, and presents personalised career recommendations along with matching college and job listings — all within a visually immersive ocean-themed interface.

Field	Details
Project Name	AI Career Compass
Type	Single-Page React Application (SPA)
Framework	React 18 via Vite build tool
Styling Approach	Inline CSS-in-JSX with CSS Variables defined in :root
Animation	Pure CSS keyframe animations — no animation library
API Integration	<code>fetch()</code> calls to FastAPI at <code>http://127.0.0.1:8000</code>
Number of Pages	5 (Auth, Input, Roadmap, Colleges, Jobs)
Fonts Used	Syne (display/headings) + DM Sans (body text) via Google Fonts
Theme	Dark ocean night — deep navy, teal, aqua, gold accents, glassmorphism cards
Responsive	Yes — CSS flexbox + grid, auto-wraps on mobile. No media queries needed.

02

Tech Stack & Tools

The frontend is built entirely with React and does not rely on any UI component library. Every component — buttons, inputs, cards, animations — is hand-coded in JSX with inline styles. This gives full design control and keeps the bundle size minimal.

Technology	Version / Source	Why We Used It
React	v18 (Vite template)	UI component model, state management, conditional rendering across 5 pages
Vite	v5	Instant dev server (<1s startup), fast HMR, replaces deprecated create-react-app
CSS Variables	Native browser	Consistent colour palette via <code>:root</code> — change one variable to retheme the whole app
CSS Keyframes	Native browser	All 16 animations — waves, islands, compass glow, shimmer text, fade-ins, ripples
Google Fonts	Syne + DM Sans	Syne for headings (bold, geometric, modern), DM Sans for body (highly readable)
fetch() API	Built into browser	HTTP POST calls to FastAPI backend — no axios or other HTTP library needed
SVG (inline)	Hand-coded JSX	Compass component + curved connecting path between career islands on roadmap
Flexbox + Grid	Native CSS	Responsive layouts — no CSS framework (Bootstrap/Tailwind) required

WHY VITE INSTEAD OF CREATE-REACT-APP?

create-react-app is officially deprecated (no longer maintained by Meta since 2023). Vite starts the dev server in under 1 second versus 30+ seconds, and hot-reloads file changes in under 50ms. The React code inside App.jsx is completely identical regardless of which tool creates the project — Vite is purely the build tooling, not the framework.

03

Application Structure & File Layout

The entire frontend lives in a single file: `src/App.jsx` . This architecture was chosen to keep the submission self-contained and easy to review. In a production codebase, each page and component would be split into separate files.

Vite Folder Structure

File / Folder	Purpose
<code>src/App.jsx</code>	MAIN FILE — entire application code lives here (all pages, components, styles)
<code>src/main.jsx</code>	Entry point — mounts <code><App /></code> into index.html. Do not edit this file.
<code>src/index.css</code>	Global CSS file — emptied out completely. All styles are defined inside App.jsx.
<code>index.html</code>	Single HTML file with <code><div id="root"></code> — React fills this with the entire app
<code>vite.config.js</code>	Vite build configuration. Default settings used — no customisation needed.
<code>package.json</code>	Lists react, react-dom, and vite as dependencies. Run <code>npm install</code> to restore.
<code>node_modules/</code>	Auto-generated installed packages. Never edit. Never commit to Git.

Internal Sections of App.jsx

Section	What It Contains	Approx Lines
API Config	API_BASE URL constant, fetch() wrapper functions	~15
Career Prep Data	CAREER_PREP object — skills, certifications, tools, tip for 24 careers	~150
Salary Benchmarks	SAL object — realistic fresher/mid/senior INR ranges for 24 careers	~40
Global Styles	CSS variables (:root), all keyframe animations, base resets via <code><style></code> tag	~120
OceanBG Component	Fixed animated background — stars, moon, 3 wave layers, bubbles, glow orbs	~70

Section	What It Contains	Approx Lines
Compass Component	SVG compass with tick marks, cardinal letters, rotating two-colour needle	~30
Reusable UI	BackBtn, Btn (ripple), FInput (floating label), TagInput, Slider	~150
AuthPage	Login + signup toggle, floating inputs, validation, success animation	~80
InputPage	3-step wizard — Academic, Skills & Interests, Preferences	~120
RoadmapPage	Floating islands, SVG path, detail panel with 4 tabs (Overview/Skills/Certs/Tools)	~160
CollegesPage	College cards, type filter pills, NIRF data, eligibility badges, score bars	~90
JobsPage	Job listings, salary benchmark card, realistic salary logic, stats row	~100
App Root	Page state machine — conditional rendering of all 5 pages	~20

04

Page-by-Page Breakdown

Page 1 — Authentication (Login / Sign Up)

The first page the user sees. Handles both login and signup in one card with a smooth toggle between modes.

Feature	Implementation Detail
Login / Signup toggle	Two-button pill toggle. Switching animates the form fields with a fadeUp animation.
Floating label inputs	FInput component — label animates upward and shrinks when the field is focused or has a value.
Password visibility toggle	Eye icon (👁️ / 👤) in the input's right side toggles type between "password" and "text".
Form validation	Checks for empty fields and password mismatch. Displays a red error box with the message.
Signup success flow	Shows green confirmation for 2.2 seconds then automatically switches back to Login tab.
Loading state	Button shows spinning circle + "Analysing..." text during the 1.1 second simulated delay.
Background	Full OceanBG component — animated stars, moon glow, 3 wave layers, rising bubbles.
After login	Calls onAuth() callback → transitions to InputPage and passes user object.
Note on auth	Authentication is simulated — any email/password is accepted. No real auth backend connected yet.

Page 2 — Input (3-Step Wizard)

A multi-step form that collects all data needed to call the /predict/detailed API endpoint. Split into 3 steps to avoid overwhelming the user.

Feature	Implementation Detail
Step 1 — Academic	Dropdown for degree type, range slider for 12th percentage (40–100%), CGPA slider (4.0–10.0).
Step 2 — Skills & Interests	TagInput component with autocomplete from predefined lists. Enter key or click to add. X button to remove. Tags stored as array, joined with ", " on submit.
Step 3 — Preferences	City dropdown (8 major cities + Remote), budget slider (₹0–25L/yr), mode toggle (Offline / Online / Distance).
Progress indicator	Three-segment bar at the top. Each segment fills with teal gradient as steps complete. Segments are clickable to jump back.
Compass rotation	Compass angle updates to step × 60 degrees — visually shows journey progression.
Get My Analysis button	Gold gradient button on Step 3. Calls api.predictDetailed() → transitions to RoadmapPage on success.
Error handling	If API is unreachable, shows red error message with the backend URL so the user knows what to check.

Page 3 — Roadmap (Career Islands)

The most visually complex page. Displays the top 3 career matches as animated floating islands on an ocean background, with a detailed preparation guide for the selected career.

Feature	Implementation Detail
Career Islands	Each island is a CSS ellipse div with gradient background, inner shadow, and iFloat animation. Each island floats at a slightly different speed and delay for an organic feel.
SVG connecting path	Inline SVG draws a curved Bezier path between all islands. Uses stroke-dasharray + drawPath animation to "draw" itself on page load.
Active island selection	Clicking an island scales it up (scale 1.08), adds a glowing animated ring, and loads that career's data in the detail panel below.
Compass rotation	Needle angle = active × 110 - 20 degrees. Updates with CSS spring transition (cubic-bezier with overshoot) each time an island is selected.

Feature	Implementation Detail
Detail panel — 4 tabs	Overview (pro tip, prep timeline, avg salary), Skills to Build (numbered list), Certifications (medal icons), Tools (chip grid). Tab switching has fadeUp animation.
Career prep data	CAREER_PREP object covers 24 careers. Fuzzy string matching handles slight name variations from the API response.
Confidence ring	SVG circle with stroke-dasharray proportional to confidence%. Animates with drawPath on career change.
CTA buttons	Two cards — Colleges (teal) and Jobs (gold) — with hover lift effect. Show count from API response data.
Back button	Top-right button returns to InputPage so user can adjust their inputs and re-run.

Page 4 — Recommended Colleges

Displays college recommendations from the API for the selected career, with type-based filtering and eligibility indicators.

Feature	Implementation Detail
College cards	Glassmorphism cards in a responsive CSS grid (auto-fill, min 370px). Cards animate in with staggered fadeUp delays.
Type filter pills	Generated dynamically from unique college types in API response. Clicking filters: IIT, NIT, IIM, AIIMS, Govt, Private.
Colour coding by type	IIT red, NIT blue, IIM purple, AIIMS green, IGNOU orange, Govt teal, Private amber badge on each card.
Eligibility tags	Eligible  (green), Reach  (gold), Aspirational  (coral) — calculated by backend from student marks vs NIRF score.
NIRF score bar	Animated progress bar (pFill animation) showing score out of 100.
Info grid inside card	2×2 grid showing location, field, fee range, and study modes in subtle background boxes.
Apply button	REMOVED — cards are display-only with no external links, as requested.
Back button	Returns to RoadmapPage.

Page 5 — Recommended Jobs

Displays fresher job listings with realistic salary data. Uses market benchmark data to fill in missing or unrealistically low salaries from the dataset.

Feature	Implementation Detail
Salary benchmark card	A highlighted card at the top shows Fresher / Mid Level / Senior salary ranges for the career. Sourced from 2024 India market data. Includes caveat note about IIT/NIT salary premium.
Realistic salary logic	If <code>salary_annual_inr</code> from API is null or below ₹1L, the career's average benchmark is substituted and labelled "Est. from benchmark". Real API salaries are labelled "Annual CTC".
Salary colour coding	Green (≥ ₹6L/yr), Gold (₹3.5–6L/yr), Coral (< ₹3.5L/yr) — helps students quickly spot higher-paying roles.
City match highlight	Jobs in the student's preferred city get a left blue border and a "Your City" badge. Sorted to appear first.
Salary bar	Each card has a gradient bar showing where that salary sits on the fresher-to-senior scale. Min and max labels shown below.
Stats row	Three cards at the top: Total Openings, Est. Average Salary, City Matches — computed from the enriched job list.
Back button	Returns to RoadmapPage.

05

Reusable Components

These components are defined once in App.jsx and reused across multiple pages. Together they form the design system of the application.

Component	Props	What It Does
OceanBG	none	Fixed-position animated background. Generates stars (twinkle animation at varied delays), moon glow (radial gradient div), 3 wave SVG layers (waveScroll at different speeds), bioluminescent orbs (glowP pulse), and rising bubbles (bubbles keyframe).
Compass	angle, size	Hand-coded SVG with 36 tick marks, N/E/S/W cardinal labels (N in gold), and a two-colour needle (coral north, translucent south). Needle rotates to the given angle with a spring CSS transition. Surrounded by compassGlow and floatA animations.
BackBtn	onClick, label	Translucent teal button with a hover translateX(-3px) micro-animation. Visible on every page except AuthPage. Makes navigation explicit.
Btn	children, onClick, loading, variant, style, disabled	Primary action button. Tracks click coordinates to spawn a ripple div at the exact click point. Three variants: primary (teal gradient), gold (amber gradient), ghost (outline only). Shows spinner + text when loading=true.
FInput	label, type, value, onChange, rightIcon	Floating-label input — label text animates up and shrinks (font size 14→11px, position top 5→-10px) on focus or when value exists. Supports rightIcon prop for password toggle eye button.
TagInput	label, value, onChange, sugg, ph	Multi-select tag input. Renders tags as removable chips (scaleIn animation on add). Shows autocomplete dropdown from sugg array filtered by current input. Enter key adds custom tags not in the list.
Slider	label, value, onChange, min, max, unit	Custom range slider. Live value shown in gold top-right. Teal gradient fill div shows proportion. Hidden native input handles interaction. CSS targets webkit thumb for consistent styling.

06

Design System — Colours, Fonts & Animations

Colour Palette (CSS Variables)

CSS Variable	Hex Value	Used For
--bg	#04101E	Page background — darkest navy, the "deep ocean" base
--surf	#071828	Slightly lighter surface for inner cards
--teal	#2EB8C8	Primary brand colour — borders, icons, progress bars, active states
--aqua	#50D4E0	Highlight colour — active states, headings, links, compass markings
--foam	#7EE8C8	Positive states — Eligible badge colour, senior salary indicator, city match
--gold	#F0B429	Secondary accent — NIRF ranks, salary figures, compass N label, gold buttons
--coral	#FF7B6B	Warning / alert — low salary colour, compass south needle, error messages
--sand	#F5E6C8	Warm subtle accent — prep timeline badge text
--txt	#E4F2F8	Primary text — all body copy and headings
--muted	rgba(190,225,240,0.42)	Secondary / placeholder text — labels, metadata, captions
--glass	rgba(6,24,46,0.68)	Glassmorphism card background — used with backdrop-filter: blur(18px)

Typography

SYNE (DISPLAY FONT)

Used for: all page headings, card titles, button labels, badge text, tab labels, island names.

DM SANS (BODY FONT)

Used for: all body paragraphs, input field text, dropdown options, table content, description

CSS variable: `--fd` . A geometric sans-serif with a wide, bold character. Weight 600–800 used.

text. CSS variable: `--fb` . Optimised for readability at small sizes with its variable optical-size axis (9–40pt).

SOURCE

Both fonts loaded via Google Fonts `@import` inside the `GlobalStyle` component at the top of `App.jsx`. No font files are bundled — they are fetched from Google's CDN on page load.

CSS Animations (All Keyframes)

Animation Name	Effect	Used On
<code>waveScroll</code>	Continuous horizontal translate (~50%)	Ocean wave layers in <code>OceanBG</code> — each wave at a different speed (20s, 15s, 11s)
<code>floatA</code>	Gentle vertical bob + slight rotation	Compass component, floating consistently on all pages
<code>iFloat</code>	Vertical float + subtle rotation	Career islands — each with different duration and delay for organic motion
<code>twinkle</code>	Opacity + scale pulse	Star particles in <code>OceanBG</code> background
<code>shimmer</code>	Gradient background-position sweep (300% travel)	Page titles — creates aqua→foam→gold→aqua sweeping gradient on text
<code>fadeUp</code>	<code>translateY(22px→0)</code> + <code>opacity(0→1)</code>	Page load reveals, tab content switches, card appearances
<code>fadeIn</code>	<code>opacity(0→1)</code>	Full page transitions — each page wraps in <code>.page</code> class
<code>scaleIn</code>	<code>scale(0.9→1)</code> + opacity	Auth card on load, tag chips appearing in <code>TagInput</code>
<code>slideR</code>	<code>translateX(-18px→0)</code> + opacity	Job cards and skill list items — staggered with <code>animation-delay</code>
<code>drawPath</code>	<code>stroke-dashoffset (800→0)</code>	SVG path between career islands, SVG confidence ring on selection

Animation Name	Effect	Used On
ripple	scale(0→3) + opacity(0.5→0)	Button click effect — div spawned at exact click coordinates
compassGlow	drop-shadow filter intensity pulse	Compass component — subtle pulsing golden glow
glowP	box-shadow intensity pulse	Active island selection ring, bioluminescent orbs in ocean
pFill	width: 0 → target width	NIRF score bars, salary bars, step progress bar fill
blink	Opacity pulse (1→0.25→1)	Node dots on SVG path between career islands
bubbles	translateY upward + scale shrink → opacity 0	Rising bubble particles in OceanBG, repeating infinitely
spin	360° rotation	Loading spinner inside Btn component when loading=true

07

API Integration with FastAPI Backend

The frontend communicates with the FastAPI backend via the browser's built-in `fetch()` API. No third-party HTTP library is used. The backend URL is defined in one constant so it can be changed in one place if the port changes.

API BASE URL

`const API_BASE = 'http://127.0.0.1:8000'` — defined at the very top of `App.jsx`. Change this value if FastAPI runs on a different port.

Endpoint Used

Endpoint	Method	Called By	Returns
<code>POST /predict/detailed</code>	POST (JSON body)	InputPage — on "Get My Analysis" click	Top 3 careers, each with <code>colleges[]</code> and <code>jobs[]</code> arrays

Request Body Fields

Field Sent	Source in Frontend
<code>skills</code>	TagInput array joined with ", " — e.g. "Python, SQL, React"
<code>interests</code>	TagInput array joined with ", " — e.g. "AI & Data, Design"
<code>marks_12th</code>	Slider value from Step 1 (range 40–100)
<code>cgpa</code>	Slider value from Step 1 (range 4.0–10.0)
<code>degree</code>	Dropdown selection from Step 1
<code>preferred_city</code>	Dropdown selection from Step 3
<code>budget_lakhs</code>	Slider value from Step 3 (0–25L)
<code>preferred_mode</code>	Toggle button from Step 3 (Offline / Online / Distance)
<code>top_n</code>	Hardcoded to 3 — always requests top 3 career recommendations

Response Data Usage

Response Field	Used In
<code>results[i].career</code>	Island name label, detail panel heading, college/jobs page title
<code>results[i].rank</code>	Island rank badge (#1, #2, #3)
<code>results[i].confidence</code>	Confidence % badge, SVG ring chart stroke-dasharray
<code>results[i].colleges[]</code>	CollegesPage — all college card data (name, type, rank, eligibility, fee, modes)
<code>results[i].jobs[]</code>	JobsPage — job title, location, city, salary_annual_inr (enriched by frontend)

CORS CONFIGURATION

The FastAPI backend already includes `CORSMiddleware` with `allow_origins=["*"]` . This allows the React dev server at `localhost:5173` to call the API at `localhost:8000` without browser security errors. Standard setup for local development.

08 Frontend Data Logic

The frontend contains two significant hardcoded datasets that enrich the API response with additional information. Both are stored as JavaScript objects inside App.jsx.

1. Career Preparation Data (CAREER_PREP)

Covers 24 career categories. For each career, the object stores:

- **skills** — list of 5 specific skills the student should develop
- **certifications** — list of 3 industry-recognised certifications to pursue
- **tools** — list of 5 software tools used professionally in that career
- **timeline** — realistic preparation time estimate (e.g. "6–12 months")
- **tip** — one actionable, specific piece of advice for that career

A `getPrep()` function does fuzzy string matching so career names returned by the API (e.g. "ML Engineer") still map to the correct entry (e.g. "ML/AI Engineer"). Falls back to a generic entry if no match is found.

2. Salary Benchmark Data (SAL)

Covers 24 career categories. For each career:

- **f** — [min, max] annual INR salary for freshers (0–1 year experience)
- **m** — [min, max] for mid-level (2–4 years experience)
- **s** — [min, max] for senior level (5+ years experience)
- **avg** — single average annual salary shown on the Roadmap detail panel

The `realSal()` function in JobsPage applies: `if (!raw || raw < 100000) return sal.avg` — if the dataset salary is missing or unrealistically low, it substitutes the benchmark average. Cards are then labelled either "Annual CTC" (real data) or "Est. from benchmark" (substituted data) so the user knows the source.

09

State Management & Page Navigation

The application uses React's built-in `useState` hook for all state. No external state management library (Redux, Zustand, Context API) is used — the project scope is small enough that callback props handle everything cleanly.

App-Level State Variables

Variable	Type	Purpose
<code>page</code>	string	Controls which page renders. Values: 'auth' 'input' 'roadmap' 'colleges' 'jobs'
<code>user</code>	object null	Stores { name, email } after login — passed to InputPage to display the greeting
<code>data</code>	object null	Full API response from /predict/detailed — passed to RoadmapPage as prop
<code>career</code>	object null	Selected career from RoadmapPage — passed to CollegesPage and JobsPage

Navigation is handled by callback props. Example: clicking "Recommended Colleges" on the Roadmap calls `onColleges(currentCareer)` which sets `career` state and transitions to 'colleges' page. There is no React Router — just conditional rendering based on the `page` string state:

NAVIGATION PATTERN (STATE MACHINE)

`{page === 'roadmap' && <RoadmapPage ... />}` — only the active page component renders. All others are unmounted from the DOM. This is a simple state-machine navigation pattern.

WHY NO REACT ROUTER?

React Router adds URL-based navigation (browser back button, bookmarkable URLs). For a project submission with 5 sequential pages that always follow the same flow, a state machine is simpler and has fewer dependencies. Adding React Router v6 later is straightforward — replace `setPage()` calls with `useNavigate()` .

10 How to Run the Full Project

Both the backend and frontend must be running at the same time. Open two separate terminal windows in VS Code (click the + icon in the terminal panel).

Terminal 1 — Start the Backend

Step	Command
Navigate to backend folder	<code>cd your-backend-folder</code> (where main.py lives)
Start FastAPI	<code>uvicorn main:app --reload</code>
Confirm it's running	Terminal shows: <code>Uvicorn running on http://127.0.0.1:8000</code>
Test in browser	Open <code>http://127.0.0.1:8000/docs</code> — FastAPI Swagger UI should appear

Terminal 2 — Start the Frontend

Step	Command
Navigate to frontend folder	<code>cd ai-career-compass</code>
Start Vite dev server	<code>npm run dev</code>
Confirm it's running	Terminal shows: <code>Local: http://localhost:5173/</code>
Open in browser	Go to <code>http://localhost:5173</code> — AI Career Compass login page appears

IMPORTANT — BOTH TERMINALS MUST STAY OPEN

Terminal 1 (backend at port 8000) and Terminal 2 (frontend at port 5173) must both be running simultaneously. Closing either terminal will break the application.

11 Likely Reviewer Questions & Answers

Study these questions carefully — they cover the most likely topics a reviewer or examiner will ask about the frontend.

Q Why React instead of plain HTML/CSS/JavaScript?

A React's component model lets us reuse components like `Btn`, `FInput`, and `TagInput` across all 5 pages without repeating code. State management with `useState` handles the multi-step form, tab switching, and page transitions cleanly. A plain HTML approach would require complex manual DOM manipulation for all these interactions.

Q Why Vite instead of create-react-app?

A `create-react-app` is deprecated and no longer maintained by Meta as of 2023. Vite starts the dev server in under 1 second versus 30+ seconds for CRA, and hot-reloads file changes in under 50ms. The React code inside `App.jsx` is completely identical regardless of which tool creates the project — Vite is just the build tool, not the framework.

Q Why no CSS framework like Tailwind or Bootstrap?

A Using inline CSS-in-JSX gives complete control over every style without fighting a framework's defaults or overrides. It also means zero external CSS dependencies. The design uses CSS variables defined in `:root` to maintain consistency — similar to how Tailwind uses a theme config. The trade-off is more verbose code, but full design flexibility.

Q Why no animation library like Framer Motion or GSAP?

A All 17 animations are done with pure CSS keyframes. This avoids adding a large dependency (Framer Motion ~100KB) for animations that native CSS handles well. CSS animations are also more performant because they run on the GPU compositor thread, not the JavaScript main thread. The trade-off is that CSS animations cannot respond to JavaScript state as fluidly as Framer Motion.

Q How does page navigation work without React Router?

A The `App` component holds a `'page'` string in state. Conditional rendering — e.g. `{page === 'roadmap' && <RoadmapPage />}` — shows only the relevant component. Navigation is triggered by callback props passed down from `App` to each page. This is a simple state machine pattern. It doesn't support browser back button or bookmarkable URLs, which is acceptable for this sequential flow.

Q What is glassmorphism and how is it implemented?

A Glassmorphism is a UI design trend where panels look like frosted glass. It is implemented with three CSS properties combined: (1) `background: rgba(6,24,46,0.68)` — semi-transparent dark fill, (2) `backdrop-filter: blur(18px)` — blurs whatever is behind the element, (3) `border: 1px solid rgba(50,180,210,0.16)` — a subtle glowing border. The `.glass` CSS class applies all three to every card.

Q How is the animated ocean background implemented?

A Three techniques combined: Stars — 55 absolutely positioned divs with random left/top positions and the `twinkle` keyframe at varied animation-duration and animation-delay values. Waves — three divs set to 200% width containing

SVG path elements, each running the waveScroll animation (translateX 0 to -50%) at different speeds (20s, 15s, 11s). Moon — a single div with radial-gradient and box-shadow. All are fixed-position behind page content.

Q What is TF-IDF and does the frontend need to know about it?

A TF-IDF (Term Frequency–Inverse Document Frequency) is the text vectorisation algorithm used by the ML model on the backend. The frontend sends a plain string of skills and interests. The backend converts it to a TF-IDF vector, runs the Logistic Regression model, and returns career predictions. The frontend is completely abstracted from the ML — it only sends input and reads output.

Q How are missing or unrealistic salaries handled in the jobs page?

A The fresher_jobs dataset has many rows where salary is null or unrealistically low (under ₹1L). The realSal() function applies: if(!raw || raw < 100000) return sal.avg. It substitutes the career's benchmark average salary from the hardcoded SAL object and labels the card "Est. from benchmark". Real API salaries are labelled "Annual CTC". A benchmark card at the top of the page also shows Fresher/Mid/Senior ranges for full context.

Q What is the compass and why does it rotate?

A The compass is a hand-coded SVG component — no image or icon library. It has 36 tick marks drawn with line elements, N/E/S/W text labels, and a two-colour needle (coral north tip, translucent south). The needle rotates via transform: rotate() driven by JavaScript: on the Input page it reflects the current step number; on the Roadmap page it updates to active × 110 - 20 degrees when a new island is selected. The transition uses cubic-bezier(0.34, 1.56, 0.64, 1) — a spring easing with overshoot — so the needle bounces slightly past the target before settling.

Q How responsive is the application?

A The layout uses CSS flexbox with flexWrap: 'wrap' for row-based sections so rows stack on small screens. Card grids use CSS grid with gridTemplateColumns: 'repeat(auto-fill, minmax(370px, 1fr))' — on narrow screens this collapses to one column automatically. No media queries are written — the layout is fluid by design. All font sizes and paddings use fixed pixel values so they may need adjustment for very small screens in a production version.

Q What is the button ripple effect and how does it work?

A When a user clicks a Btn component, a click event handler reads e.clientX and e.clientY (mouse position) and calculates the position relative to the button element using getBoundingClientRect(). A small div is then absolutely positioned at that exact point inside the button and given the ripple CSS animation (scale 0 to 3, opacity 0.5 to 0). The div is removed from state after 700ms. This creates a Material Design-style ripple that originates from the actual click point.

12

Known Limitations & Future Improvements

Current Limitation	How to Improve in Future
Authentication is simulated — any email/password is accepted	Connect to a real auth backend using FastAPI + JWT tokens, or integrate Firebase Authentication
All code is in one App.jsx file — hard to maintain at scale	Split into separate files: /components/Compass.jsx, /pages/RoadmapPage.jsx, /data/careerPrep.js, etc.
No URL-based navigation — browser Back button does not work	Add React Router v6 — replace setPage() calls with useNavigate() hooks
Career prep data and salary benchmarks are hardcoded in the file	Move to a backend database or static JSON API endpoint — update without redeploying frontend
Salary data is 2024 India market estimates — may become outdated	Connect to a live salary API such as AmbitionBox, Glassdoor, or LinkedIn Salary Insights
No user profile persistence — analysis lost on page refresh	Add localStorage to save the last analysis, or build a backend user profile system
No dark/light mode toggle	Add theme toggle using a data-theme attribute on body and two sets of CSS variable values
Mobile touch gestures not implemented for island selection	Add swipe-left/right navigation between career islands using touch events or react-swipeable

AI Career Compass — Frontend Documentation

Career Guidance System Project | Built with React + Vite | Connects to FastAPI ML Backend

Open this file in any browser · Use the "Save as PDF" button (bottom-right) to save a PDF copy